



Chương 5: Kiến trúc bộ nhớ trong hệ thống nhúng

Computer Engineering Faculty

Embedded Systems

Nội Dung



1

Các loại bộ nhớ

2

Ảnh xạ bộ nhớ

3

Tổ chức bộ nhớ

4

Mô hình bộ nhớ trong C

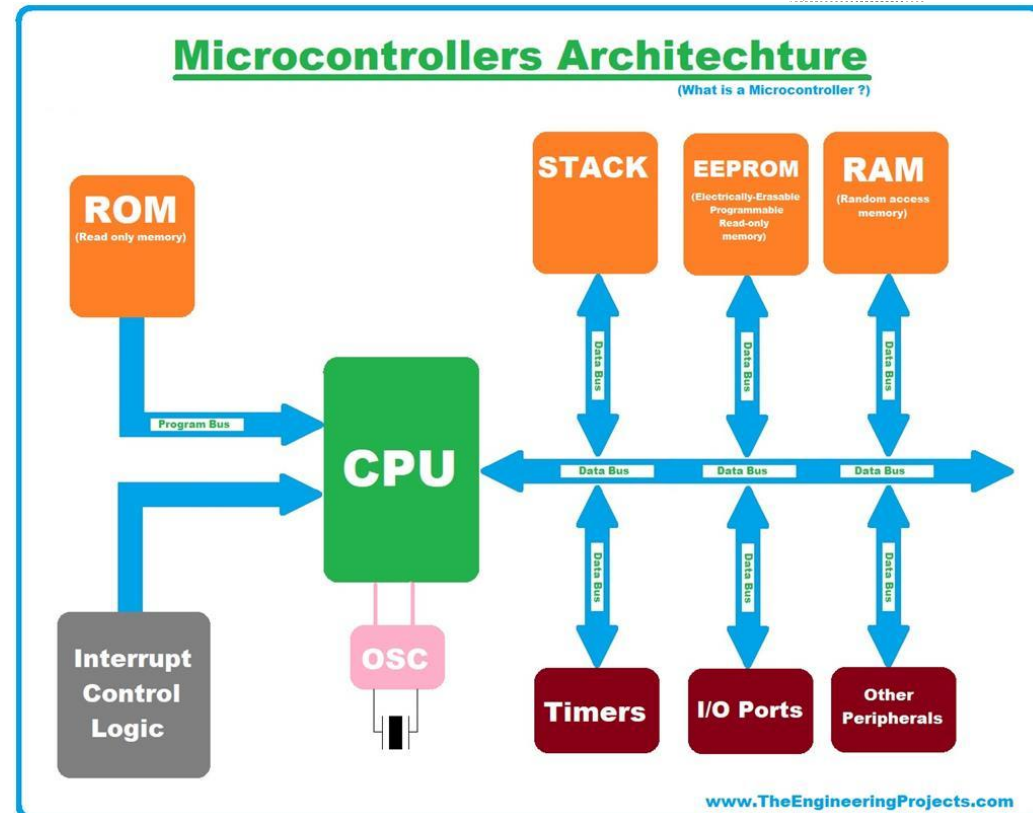
5

Phân cấp bộ nhớ



Các loại bộ nhớ

- ❑ **ROM** (Read Only Memory):
 - Chứa chương trình khởi động, firmware.
- ❑ **RAM** (Random Access Memory):
 - dữ liệu tạm, volatile.
- ❑ **Flash/EEPROM**:
 - lưu dữ liệu lâu dài, tái lập trình được.
- ❑ **Register**:
 - lưu trữ tốc độ cao bên trong CPU.
- ❑ **Ví dụ ứng dụng**:
 - bootloader trong ROM,
 - biến chương trình trong RAM.



Ánh xạ bộ nhớ



□ Khái niệm:

- Gán địa chỉ logic → địa chỉ vật lý.

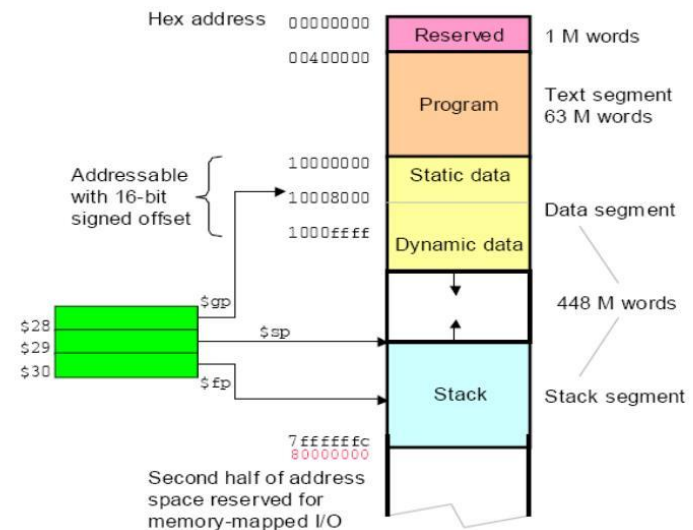
□ Kiểu ánh xạ:

- Flat memory (địa chỉ liên tục).
- Segmented memory.

□ Ý nghĩa:

- Quyết định cách CPU truy xuất dữ liệu & chương trình.

Memory Map





Tổ chức bộ nhớ

❑ Stack:

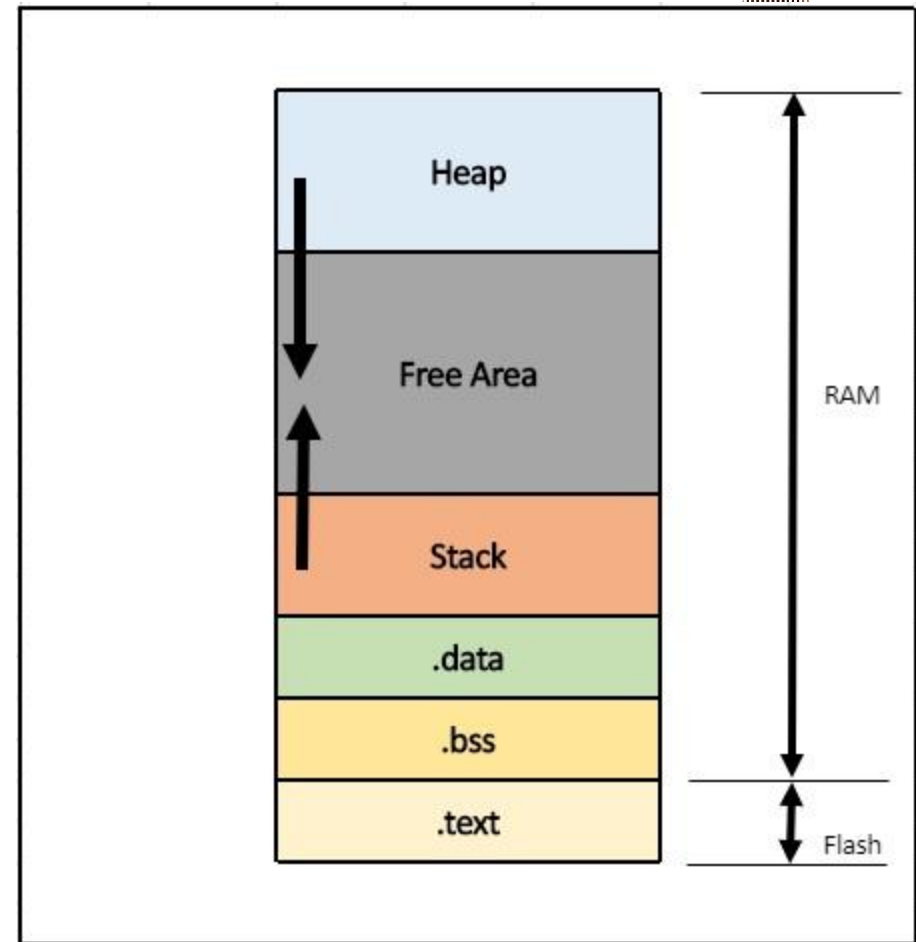
- Quản lý theo LIFO, dùng cho hàm & biến cục bộ.

❑ Static/Data segment:

- Biến toàn cục, biến tĩnh.

❑ Heap:

- Cấp phát động (malloc, free).



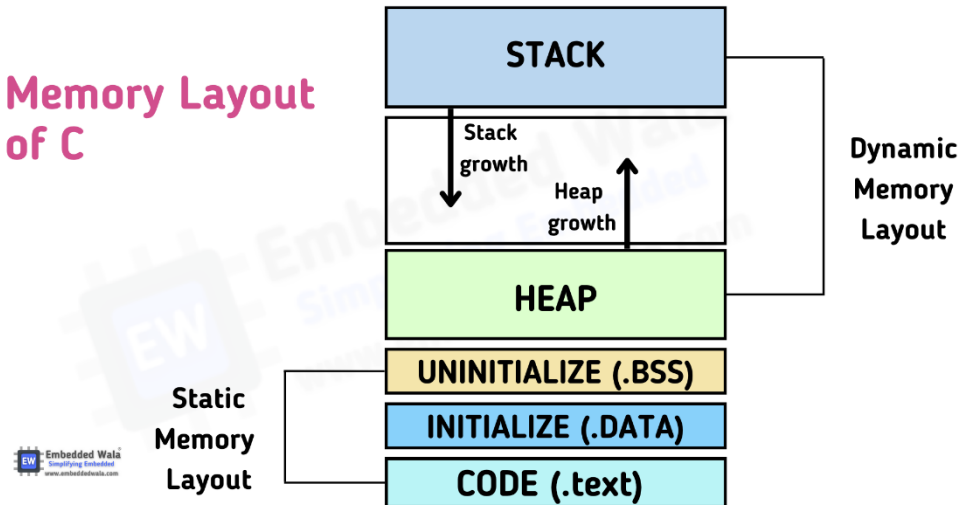


Mô hình sử dụng bộ nhớ trong C

□ Phân chia trong chương trình C:

- Text (code segment).
- Data (biến toàn cục).
- BSS (biến chưa khởi tạo).
- Heap (cấp phát động).
- Stack (gọi hàm, biến cục bộ).

Memory Layout of C





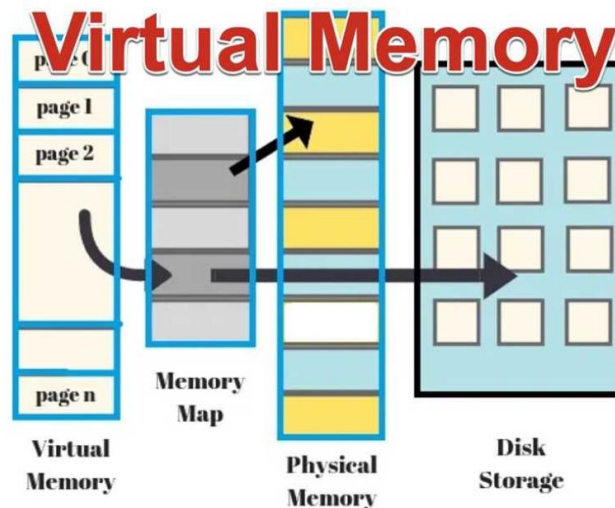
Phân cấp bộ nhớ: cache & bộ nhớ ảo

❑ Cache:

- Bộ nhớ tốc độ cao giữa CPU và RAM.
- Giảm độ trễ truy cập.

❑ Bộ nhớ ảo (Virtual Memory):

- Tách biệt địa chỉ logic và địa chỉ vật lý.
- Hỗ trợ đa nhiệm, bảo vệ bộ nhớ.





Các vấn đề thường gặp

❑ Tràn bộ nhớ Stack (Stack Overflow)

- Nguyên nhân:
 - Gọi hàm đệ quy quá sâu.
 - Dùng mảng lớn trong biến cục bộ.
- Hậu quả: ghi đè dữ liệu, chương trình treo hoặc reset.
- Ví dụ: `char buf[1024];` khai báo trong hàm trên STM32 (stack chỉ vài KB).

❑ Quản lý Heap không đúng (Heap Fragmentation)

- Nguyên nhân:
 - Cấp phát động (`malloc/free`) nhiều lần, kích thước thay đổi.
 - Không giải phóng bộ nhớ.
- Hậu quả: bộ nhớ bị phân mảnh, dù còn RAM nhưng không cấp phát được.
- Ví dụ: dùng `malloc` trong vòng lặp mà không `free` → leak RAM → chương trình chết sau vài phút.



Các vấn đề thường gặp

❑ Sử dụng sai vùng nhớ

- Nguyên nhân:
 - Nhầm lẫn Flash vs RAM.
 - Ghi dữ liệu vào Flash trong khi chưa erase.
- Hậu quả: ghi lỗi, dữ liệu hỏng, MCU treo.
- Ví dụ: ghi biến runtime vào vùng const trong Flash.

❑ Tràn bộ đệm (Buffer Overflow)

- Nguyên nhân:
 - Viết quá số phần tử mảng.
 - Không kiểm tra độ dài khi copy chuỗi (`strcpy`, `sprintf`).
- Hậu quả: ghi đè stack/heap, lỗi ngẫu nhiên khó debug.
- Ví dụ: `char buf[10]; sprintf(buf, "Hello world");` → overflow.



Các vấn đề thường gặp

❑ Sai địa chỉ con trỏ (Dangling Pointer / NULL Pointer)

- Nguyên nhân:
 - Dùng con trỏ chưa khởi tạo.
 - Truy xuất sau khi `free`.
- Hậu quả: crash, hành vi không xác định.
- Ví dụ:

```
char *p;  
*p = 'a';    // p chưa trỏ đến đâu cả
```

❑ Không tính đến giới hạn bộ nhớ nhỏ

- Nguyên nhân: copy nguyên code từ PC xuống MCU.
- Hậu quả: code quá nặng, không vừa Flash/RAM.
- Ví dụ: đưa thư viện `printf` chuẩn vào chip 32KB Flash → hết dung lượng.



Các vấn đề thường gặp

- ❑ **Race Condition khi dùng bộ nhớ chia sẻ (ISR vs main loop)**
 - **Nguyên nhân:**
 - Biến toàn cục bị truy cập đồng thời bởi ngắt (ISR) và main.
 - **Hậu quả:** dữ liệu sai, lỗi ngẫu nhiên.
 - **Ví dụ:** biến counter tăng trong ISR nhưng đọc trong main mà không `volatile`.
- ❑ **Thiếu cơ chế bảo vệ bộ nhớ (Memory Protection)**
 - MCU nhỏ không có MMU → dễ ghi nhầm vùng nhớ quan trọng.
 - Lỗi ghi tràn có thể ghi đè lên vector ngắt, làm hệ thống reset liên tục.



Ví dụ

□ Ví dụ 1:

- Chương trình C quản lý stack/heap khi gọi hàm đệ quy.

□ Ví dụ 2:

- So sánh hiệu suất khi có cache vs không có cache.

□ Thảo luận:

- Vì sao trong hệ thống nhúng thường hạn chế sử dụng heap?
- Trường hợp nào cần dùng bộ nhớ ảo?



UIT-HCM



Thank You !